

PROJECT

Project-Qwen3.5-4B-QLoRA-Coding

Fine-Tuning Qwen3.5-4B-Base for Python & ML/Data Science Coding using QLoRA

Detailed Technical Report

Item	Configuration
Base model	Qwen/Qwen3.5-4B-Base
Adaptation method	4-bit QLoRA + LoRA SFT
Dataset source	sahil2801/CodeAlpaca-20k
Final dataset	5,338 curated examples
Training split	4,270 examples
Validation split	534 examples
Held-out test split	534 examples
Hardware	NVIDIA RTX 4050 Laptop GPU — 6 GB VRAM
System RAM	16 GB DDR5
Operating system	Windows / PowerShell
Python	3.11.9
Final status	Training complete + initial held-out evaluation complete

Prepared as a portfolio-quality technical record. The report separates measured results from conclusions that require further validation.

Contents

- 1. Executive Summary
- 2. Research Question and Objectives
- 3. Hardware and Software Environment
- 4. Model Selection and QLoRA Method
- 5. Dataset Acquisition and Raw Audit
- 6. Dataset Curation and Domain Selection
- 7. Final Dataset Construction and Leakage Checks
- 8. Token-Length Investigation
- 9. Qwen3.5 Architecture Inspection
- 10. LoRA Configuration and Verification
- 11. Training Configuration
- 12. Memory Engineering and Sequence-Length Decision
- 13. Dry-Run Validation
- 14. Failure and Debugging Record
- 15. Final Training Run and Metrics
- 16. Evaluation Methodology
- 17. Generation Evaluation
- 18. Functional Execution Evaluation
- 19. Evidence Screenshot
- 20. Interpretation, Limitations, and Next Experiments
- 21. Reproducibility Checklist
- 22. Final Portfolio Summary

1. Executive Summary

This project investigates whether Qwen3.5-4B-Base can be adapted into a more Python- and ML/Data-Science-oriented coding model using parameter-efficient fine-tuning on a consumer laptop with only 6 GB of GPU VRAM.

The work was intentionally treated as an engineering experiment rather than a simple model-training exercise. The process included environment validation, 4-bit model loading, baseline generation, dataset auditing, heuristic domain scoring, train/validation/test splitting, prompt-leakage checks, architecture inspection, LoRA target selection, dry-run training, memory stress testing, failure recovery, full QLoRA training, deterministic generation evaluation, and execution-based functional evaluation.

The final run completed 534 training steps for one epoch. It trained 32,464,896 LoRA parameters, approximately 0.766% of the 4.238B-parameter model. Peak allocated VRAM during the final run was 5.443 GB, demonstrating that the selected 256-token configuration could run within the 6 GB hardware constraint.

On a 50-example functional evaluation sample drawn from the 534-example held-out test set, the base model produced outputs that the evaluator could successfully execute in 11/50 cases (22.0%), while the QLoRA model reached 44/50 (88.0%). This is a substantial executability improvement, but it must not be described as 88% semantic code accuracy.

Key measured outcomes

Measurement	Observed result
Raw dataset	20,022 records
Final curated dataset	5,338 records
Train / validation / test	4,270 / 534 / 534
LoRA trainable parameters	32,464,896
Trainable fraction	0.766%
Final training loss	0.08355
Final validation loss	~0.3977
Peak allocated VRAM	5.443 GB
Functional execution — base	22.0%
Functional execution — QLoRA	88.0%

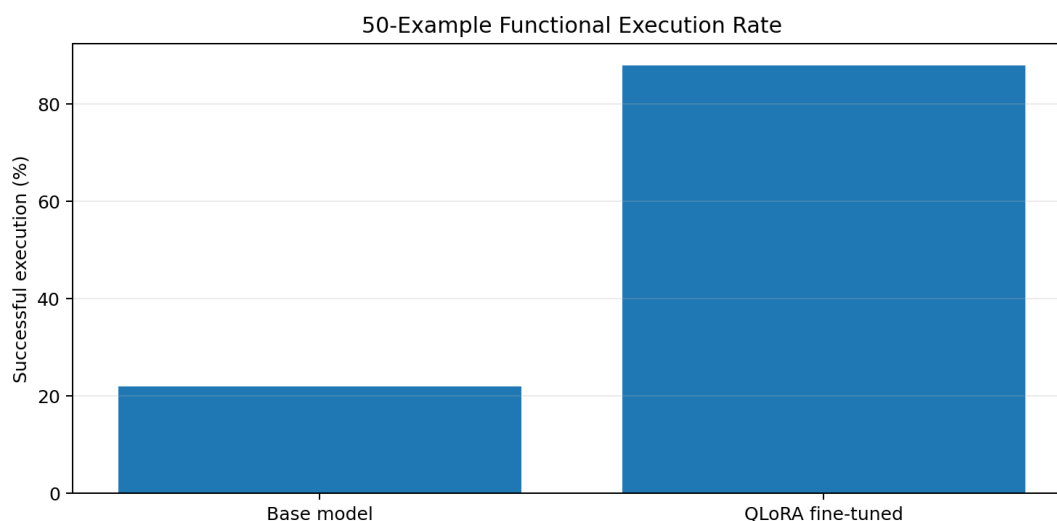


Figure 1. Aggregate functional execution rate on the 50-example evaluation sample.

2. Research Question and Objectives

Primary engineering question: Can a 4B-parameter coding-capable base model be adapted to Python and ML/Data Science coding tasks using QLoRA on an RTX 4050 Laptop GPU with 6 GB VRAM, while keeping the process reproducible and measuring whether the adaptation changes coding behavior?

The project was structured around five objectives:

- Establish a reproducible baseline for Qwen3.5-4B-Base before fine-tuning.
- Construct a curated coding dataset with an explicit Python + ML/Data Science focus.
- Apply 4-bit QLoRA with a small trainable parameter footprint.
- Determine a sequence length that is actually runnable on the 6 GB GPU rather than relying on theoretical settings.
- Compare the base and adapted models on held-out prompts using generation and execution-oriented measurements.

Scope boundary

The experiment is not intended to establish universal superiority of the fine-tuned model. The evidence is specific to the selected dataset, training configuration, hardware, generation protocol, and evaluation samples. External benchmark performance and semantic correctness remain separate questions.

3. Hardware and Software Environment

Hardware

Component	Configuration
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
GPU memory	6 GB
System memory	16 GB DDR5
Operating system	Windows
Shell	PowerShell
Python	3.11.9
NVIDIA driver	591.59
nvidia-smi CUDA report	13.1
PyTorch CUDA build	13.0

Verified software stack

Package	Version
PyTorch	2.12.1+cu130
Transformers	5.16.1
Datasets	5.0.1
PEFT	0.20.0
TRL	1.12.0
Accelerate	1.14.0
bitsandbytes	0.50.2
safetensors	0.8.0
tokenizers	0.23.1
pandas	3.0.5
NumPy	2.4.6
requests	2.34.2

Environment validation sequence

- PyTorch and CUDA availability check.
- NumPy compatibility check.
- Transformers, datasets, PEFT, TRL and Accelerate stack verification.
- bitsandbytes CUDA/NF4 kernel validation.
- Standalone 4-bit Qwen3.5-4B model loading.
- Tokenizer and chat-template inspection.

- Baseline generation before any adapter training.

The environment was validated incrementally so that a multi-hour training run would not also be the first integration test.

4. Model Selection and QLoRA Method

Base model: **Qwen/Qwen3.5-4B-Base**. The experiment starts from a base model rather than an already specialized instruction model because the research question concerns whether a relatively compact base model can be adapted to a focused coding distribution.

4-bit model loading

The final quantization configuration was:

```
BitsAndBytesConfig( load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype=torch.float16,
bnb_4bit_use_double_quant=True, )
```

The standalone 4-bit load completed successfully after a transient Windows native crash was resolved by rebooting. The successful load reported approximately 2.906 GB allocated VRAM during model loading.

Why QLoRA?

Full fine-tuning updates billions of model parameters and is not appropriate for a 6 GB laptop GPU. QLoRA instead keeps the quantized base model frozen and learns a comparatively small set of low-rank adapter parameters. This dramatically reduces the trainable parameter and optimizer-memory footprint.

QLoRA Adaptation Concept

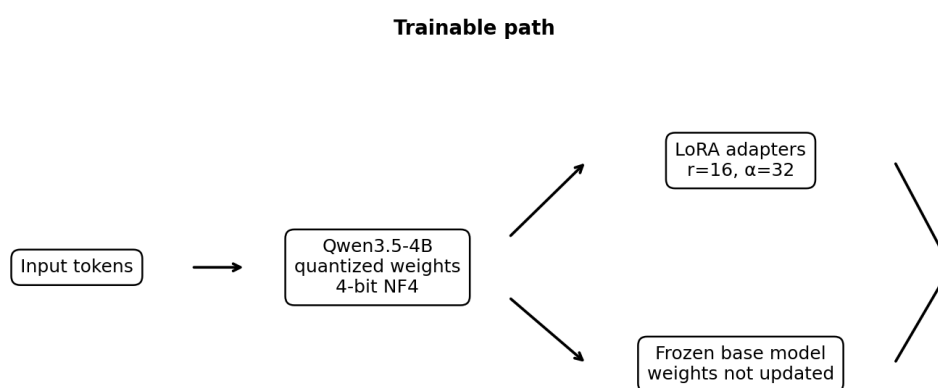


Figure 2. Conceptual QLoRA structure used in the experiment.

Adaptation path

The final setup used 4-bit NF4 quantization, double quantization, FP16 compute for the quantized model, and LoRA adapters attached to all linear modules. The trainer then optimized the adapter parameters using supervised fine-tuning.

5. Dataset Acquisition and Raw Audit

Source: **sahil2801/CodeAlpaca-20k**. The raw dataset contains instruction/input/output fields and approximately 20k synthetic coding examples.

During acquisition, the Hugging Face dataset service returned an HTTP 429. The repository JSON file was therefore downloaded directly. The final local source contained 20,022 records.

Raw quality audit

Audit field	Observed value
Total records	20,022
Empty instructions	0
Empty inputs	9,764
Empty outputs	6
Duplicate records	0
Instruction minimum / median / mean / max	14 / 71 / 73.8 / 288 chars
Input minimum / median / mean / max	0 / 5 / 23.4 / 635 chars
Output minimum / median / mean / max	0 / 131 / 197.0 / 3,905 chars
Outputs under 20 characters	1,069

The 9,764 empty inputs were not treated as corruption because the dataset schema allows an instruction to stand without an additional context field. Six records with empty outputs were removed.

Short outputs were deliberately not removed solely because they were short. A concise coding answer can be valid, so output length was treated as an audit signal rather than a binary quality label.

Initial language analysis

Detected category	Approx. count
Python	7,967
SQL	1,892
JavaScript	1,670
Java	746
C/C++	462
ML/Data	421

6. Dataset Curation and Domain Selection

The project initially considered a FinTech-oriented coding specialization. A broad finance keyword filter produced thousands of candidates, but manual inspection showed that the result was dominated by false positives. Words such as 'return', 'value', 'rate', and 'capital' occur naturally in programming tasks and are not reliable indicators of finance content.

Stricter domain scoring

Scoring rule	Count
Python score ≥ 3	5,731
ML score ≥ 4	338
ML score ≥ 7	175
Finance score ≥ 4	25
Finance score ≥ 6	1
Finance score ≥ 8	1

The strict scoring showed that the available high-confidence finance/FinTech subset was too small to support a meaningful specialization. The project therefore explicitly adopted Python + ML/Data Science coding as its training domain rather than making a stronger specialization claim than the source data could support.

Final selection strategy

- Select 338 high-confidence ML/Data examples using the selected ML score threshold.
- Select 5,000 general Python examples using the Python score threshold while excluding the selected ML examples.
- Shuffle the combined dataset using seed 42.
- Split the resulting 5,338 examples into train, validation and held-out test sets.

Final composition

Component	Examples	Approx. share
General Python	5,000	93.7%
ML/Data Science	338	6.3%
Total	5,338	100%

7. Final Dataset Construction and Leakage Checks

After cleaning and scoring, the final dataset construction produced:

Stage	Result
Raw records	20,022
Empty outputs removed	6
Remaining records	20,016
Duplicate prompt/input pairs	0
Final selected examples	5,338
Training examples	4,270
Validation examples	534
Test examples	534

Leakage verification

Prompt/input overlap checks were performed pairwise across the three splits. The result was zero overlap for train↔validation, train↔test, and validation↔test. This is important because a held-out evaluation is only meaningful if the model has not effectively seen the same prompt during training.

JSONL representation

```
{ "messages": [ {"role": "user", "content": "..."}, {"role": "assistant", "content": "..."} ] }
```

The final conversational JSONL representation is compatible with the TRL SFT workflow used for training. Assistant-only loss was enabled so that the training objective focused on the assistant response rather than the user prompt.

8. Token-Length Investigation

Token length became a central engineering issue because the raw dataset contains long examples while the GPU has only 6 GB VRAM. A first token-audit implementation was found to be incorrect: it measured the length of a structured tokenizer return object and repeatedly produced the value 2. The error was recognized and corrected.

Corrected audit logic

The corrected process first rendered the conversational example through the Qwen chat template without tokenization, then tokenized that final text and measured the length of the actual `input_ids` sequence.

Observed longest training example: approximately 1,786 tokens.

This finding motivated the dedicated memory stress test rather than assuming that a 512-token sequence length would fit simply because the model itself could load in 4-bit.

Family	Examples
Conventional attention	q_proj, k_proj, v_proj, o_proj
MLP	gate_proj, up_proj, down_proj
Hybrid / linear attention	in_proj_qkv, in_proj_z, in_proj_b, in_proj_a, out_proj
Output	lm_head

9. Qwen3.5 Architecture Inspection

Before selecting LoRA target modules, the model was inspected programmatically. The model exposed 249 nn.Linear modules and a hybrid architecture containing both conventional attention projections and additional linear-attention projections.

Observed projection families

This inspection affected the LoRA design. A narrow q_proj/v_proj target would not cover the full set of linear transformations present in the architecture. The experiment therefore used target_modules='all-linear'.

10. LoRA Configuration and Verification

Final adapter configuration

```
LoraConfig( r=16, lora_alpha=32, lora_dropout=0.05, bias="none", target_modules="all-linear",  
task_type="CAUSAL_LM", )
```

Parameter	Value
Targeted modules	248
Trainable parameters	32,464,896
Total parameters	4,238,216,192
Trainable fraction	0.766%
Rank	16
Alpha	32
Dropout	0.05
Targeting	all-linear

The adapter verification established that LoRA was actually attached to the intended modules before full training. Only 0.766% of the total model parameters were trainable.

Why this matters

A successful model load alone does not prove that the intended parameters are being optimized. The explicit trainable-parameter check verifies the adapter footprint and provides a reproducibility anchor for the final experiment.

11. Training Configuration

Setting	Final value
Model	Qwen/Qwen3.5-4B-Base
Max sequence length	256
Per-device train batch	1
Per-device eval batch	1
Gradient accumulation	8
Effective batch size	8
Epochs	1
Learning rate	1e-4
Weight decay	0.01
Warmup steps	16
Scheduler	cosine
Optimizer	paged_adamw_8bit
Gradient checkpointing	enabled
Reentrant checkpointing	False
Packing	disabled
Assistant-only loss	enabled
Seed / data seed	42 / 42
Checkpoint retention	2
Dataloader workers	0
Hub push	disabled

Training objective

The trainer performed supervised fine-tuning over conversational examples. The base model remained quantized/frozen while gradients were applied to the LoRA adapter parameters.

Final adapter location: model/qwen3.5-4b-python-qlora.

12. Memory Engineering and Sequence-Length Decision

The 6 GB VRAM limit made sequence length a practical constraint. Rather than choosing a maximum length from convenience, two configurations were experimentally compared.

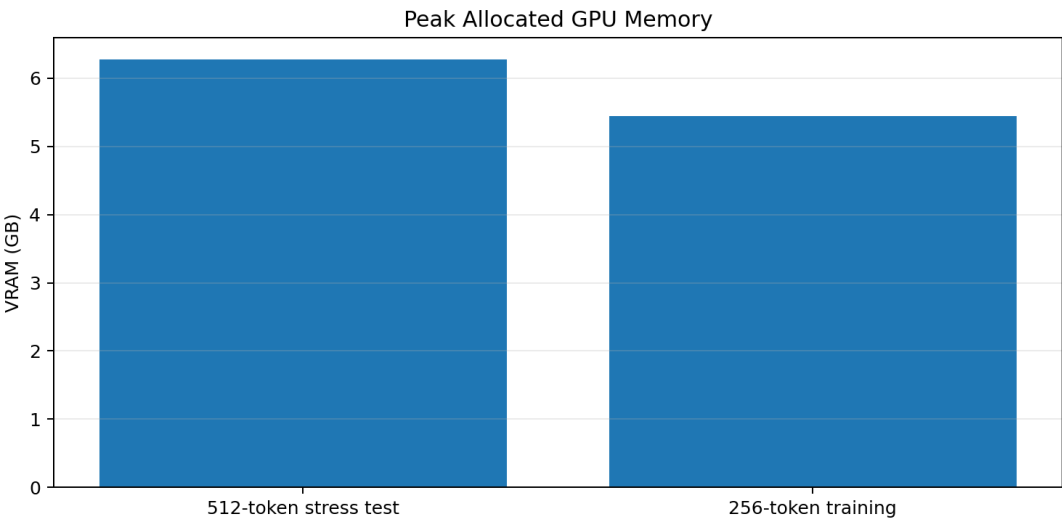


Figure 3. Peak allocated VRAM for the rejected 512-token stress test versus the final 256-token configuration.

Test	Peak allocated	Peak reserved	Decision
512-token stress test	6.279 GB	6.604 GB	Rejected — exceeds 6 GB
256-token final training	5.443 GB	5.756 GB	Accepted

The 512-token test exceeded the physical GPU capacity. The 256-token configuration still operated close to the limit but completed successfully. This is why 256 was selected as the final training sequence length.

This decision also introduces a limitation: some long training examples and long evaluation responses cannot be represented in full under the 256-token training context or the later generation constraints. The report therefore treats sequence length as both a hardware engineering parameter and an evaluation limitation.

13. Dry-Run Validation

Before committing to the full run, one real training example was passed through the QLoRA training pipeline. The test was designed to validate the actual training mechanics rather than merely importing the libraries.

Validated operations

- Model and adapter initialization.
- Forward pass through the quantized base plus LoRA modules.
- Backward pass and gradient creation.
- Optimizer step.
- GPU memory behavior.

Dry-run peak allocated VRAM: 4.799 GB.

The dry run passed, providing a lower-cost integration check before the complete 4,270-example training run.

14. Failure and Debugging Record

The final result depended on several debugging cycles. These are documented because they are part of the technical evidence rather than noise to be removed from the project history.

Failure 1 — Windows native access violation

One model-loading attempt terminated with exit code -1073741819 (0xC0000005), a Windows access-violation/native crash. The failure occurred during model loading. After a system reboot, the standalone 4-bit load completed successfully.

Failure 2 — Early training interruption

An early run stopped around step 8. No valid resumable checkpoint existed, so the run could not be safely continued from saved trainer state.

Failure 3 — Incomplete checkpoint

A later crash created a checkpoint-25 directory, but `trainer_state.json` was missing. The directory was therefore treated as invalid for automatic resume and was removed rather than being presented as a valid checkpoint.

Failure 4 — Mixed-precision scaler error

A trainer configuration using `fp16=True` failed at the first optimizer step with an `_amp_foreach_non_finite_check_and_unscale_cuda NotImplementedError` for `BFloat16`. The final trainer configuration disabled trainer-level `fp16` and `bf16` while retaining `FP16` as the 4-bit model's compute dtype.

Resulting engineering practice

The training script was made restart-aware and only treated checkpoints containing `trainer_state.json` as safely resumable. This reduced the risk of silently resuming from an incomplete state.

15. Final Training Run and Metrics

The final run completed the full 534-step training schedule for one epoch.

Metric	Final result
Progress	534 / 534
Epoch	1.0
Training runtime	5,318.7316 seconds
Training samples/sec	0.803
Training steps/sec	0.100
Total FLOPs estimate	1.0425e16
Final training loss	0.0835488
Validation loss	~0.3977
Validation token accuracy	~0.8814
Peak allocated VRAM	5.443 GB
Peak reserved VRAM	5.756 GB

Training versus validation

The final training loss is substantially below the validation loss. This gap is not automatically evidence of failure, but it is a signal that the model fit the training distribution strongly enough that held-out evaluation is necessary. Token-level loss and accuracy alone cannot establish executable-code correctness.

Final adapter artifact

The final adapter was saved locally under `model/qwen3.5-4b-python-qlora`. Large model and checkpoint artifacts were intentionally excluded from GitHub so the public repository remains source- and documentation-focused.

16. Evaluation Methodology

Evaluation compares the untouched Qwen3.5-4B-Base model against the same base model with the trained LoRA adapter. The evaluation uses held-out test data that was not included in fine-tuning.

Controlled generation conditions

- Same held-out prompts.
- Same tokenizer and chat template.
- Thinking disabled.
- Deterministic decoding with sampling disabled.
- Recorded input/output token counts and generation timing.
- Reference answers retained for later analysis.

Two separate 50-example evaluations

The generation evaluation and functional execution evaluation were run as separate 50-example samples from the 534-example held-out test set. They should therefore not be described as the exact same 50 prompts.

Why execution was included

Token-level metrics can reward textual similarity without establishing that a generated program works. Executability is a closer functional signal for coding tasks, although it still falls short of semantic correctness.

17. Generation Evaluation

Configuration

Setting	Value
Held-out test set	534 examples
Evaluation sample	50 examples
Maximum input tokens	256
Maximum new tokens	1,024
Sampling	False
Thinking	False
Decoding	Deterministic

Aggregate generation results

Metric	Base	QLoRA
Mean generation time	62.69 s	219.82 s
Median generation time	63.12 s	232.32 s
Mean throughput	7.89 tok/s	4.38 tok/s
Reached 1,024-token ceiling	6/50 (12%)	46/50 (92%)

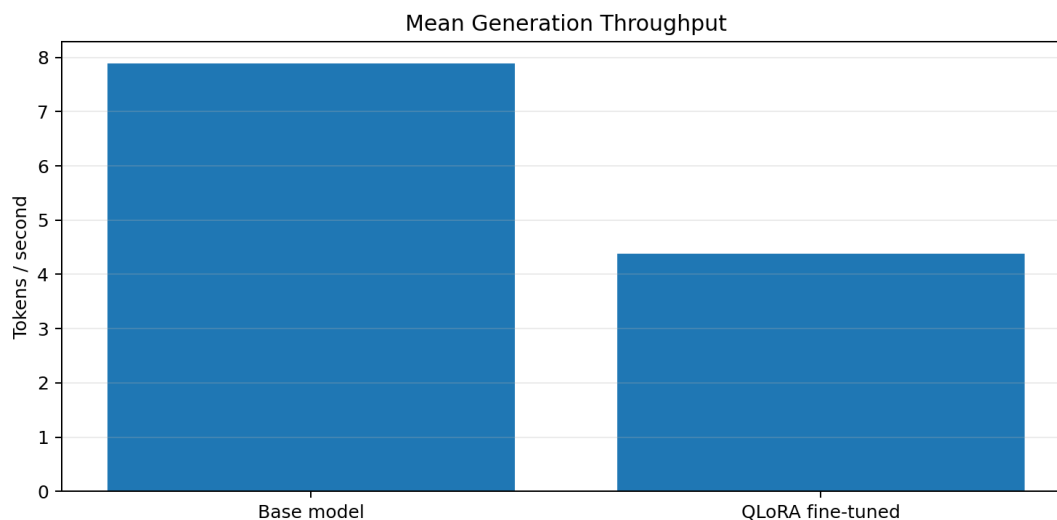


Figure 4. Mean generation throughput under the evaluation configuration.

The fine-tuned model was slower in tokens/second under this test and also generated substantially longer outputs. Because it reached the 1,024-token ceiling in 92% of sampled cases, raw wall-clock generation time is not an isolated measure of model quality or efficiency.

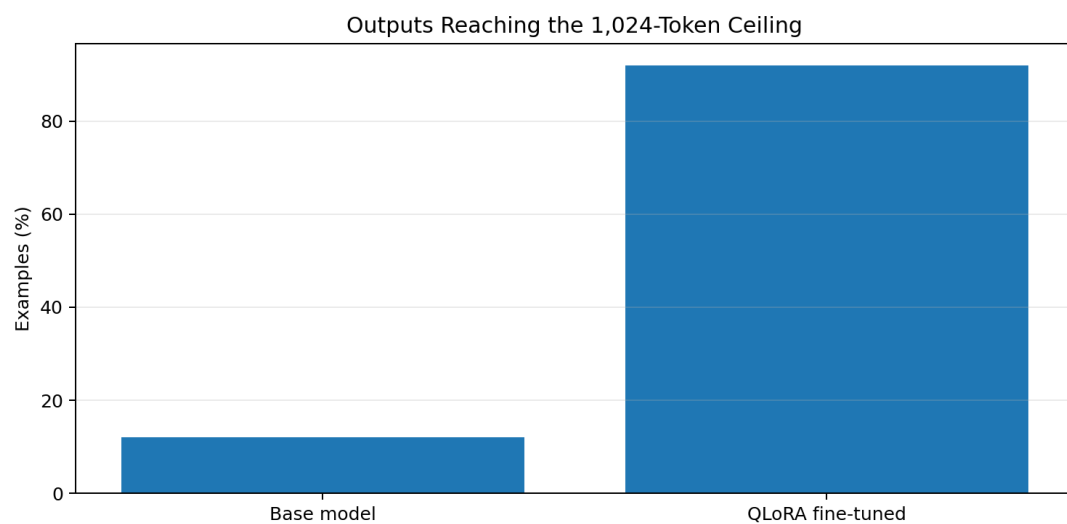


Figure 5. Fraction of sampled outputs reaching the 1,024-token generation ceiling.

18. Functional Execution Evaluation

A separate 50-example held-out sample was processed through the project's execution evaluator. The evaluator categorized generated outputs into outcomes such as `executed`, `syntax_error`, `no_code`, `no_function`, `runtime_error`, and `timeout`.

Final aggregate result

Metric	Base	QLoRA
Successfully executed	11 / 50	44 / 50
Execution rate	22.0%	88.0%
Non-successful cases	39 / 50	6 / 50

Observed behavior

The base model frequently produced explanatory prose, Markdown code blocks, or outputs that the evaluator could not classify as directly executable. The fine-tuned model more often produced directly executable Python. The per-example log showed many conversions from `syntax_error/no_code/no_function` to `executed`.

The fine-tuned sample still contained failures, including syntax errors, a runtime error, a timeout, and a no-code case. Therefore the model did not become universally executable.

Interpretation

The measured change from 22.0% to 88.0% is best described as an **executability improvement** on the sampled held-out tasks. It should not be rewritten as an 88% coding-accuracy claim.

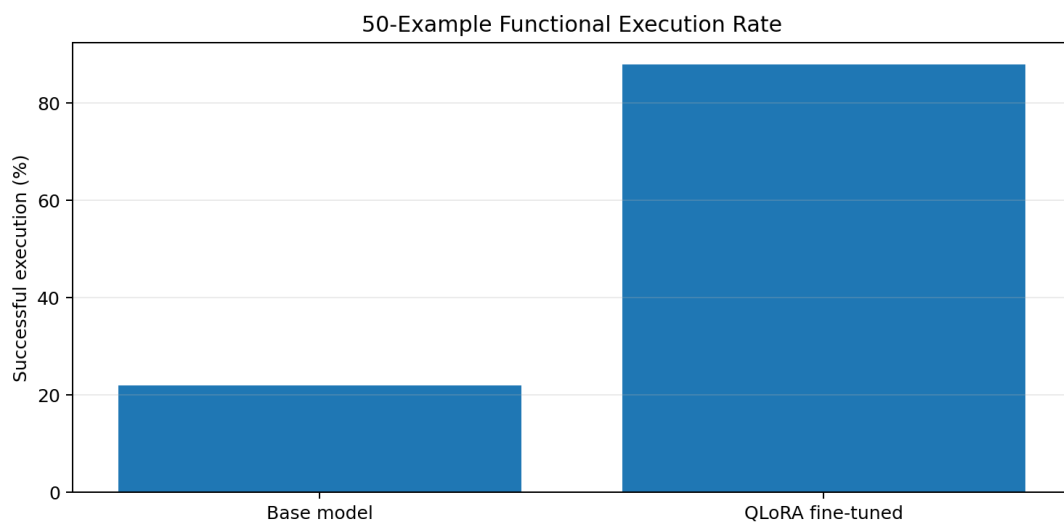


Figure 6. Base versus QLoRA functional execution rate.

19. Evidence Screenshot

The following terminal capture is the official evidence for the completed 50-example functional evaluation. It records the held-out set size, sample size, aggregate execution counts, output files, and completion marker.

```
[48/50] Test index 125
Base: syntax_error
Fine-tuned: executed

[49/50] Test index 213
Base: syntax_error
Fine-tuned: executed

[50/50] Test index 270
Base: no_code
Fine-tuned: executed

=====
EXECUTION EVALUATION COMPLETE
=====

Held-out test set: 534
Execution sample: 50

Base executed successfully: 11/50 (22.0%)
Fine-tuned executed successfully: 44/50 (88.0%)

Files:
evaluation/execution_results.json
evaluation/execution_summary.json
evaluation/execution_results.csv

FINAL FUNCTIONAL TEST COMPLETE
(.venv) PS C:\Users\rageha\Project02-ML-Coding> |
```

Figure 7. Official terminal evidence: held-out set 534; execution sample 50; Base 11/50 (22.0%); QLoRA 44/50 (88.0%).

Report evidence mapping

Evidence stage	Purpose
Environment validation	Confirms CUDA/GPU/software stack
4-bit model load	Confirms model fits for QLoRA initialization
Architecture inspection	Supports all-linear LoRA target decision
LoRA verification	Confirms trainable parameter footprint
Dry run	Confirms forward/backward/optimizer pipeline
512-token stress test	Supports 256-token hardware decision
Training completion	Confirms full run and final VRAM usage
50-example functional test	Supports executability comparison

20. Interpretation, Limitations, and Next Experiments

What the experiment establishes

- Qwen3.5-4B-Base can be loaded in 4-bit NF4 on the RTX 4050 configuration used here.
- LoRA adapters can be attached across the selected linear modules with only 0.766% of model parameters trainable.
- A curated 5,338-example Python + ML/Data dataset can be prepared with zero detected prompt/input overlap across splits.
- A one-epoch QLoRA training run can complete within the 6 GB VRAM constraint when sequence length is limited to 256 tokens.
- On the sampled functional evaluation, the fine-tuned model produced outputs that were executable far more often than the base model.

What the experiment does not establish

- 88% semantic code correctness.
- Improvement on HumanEval or MBPP.
- General superiority on real-world software engineering.
- Generalization beyond the CodeAlpaca-derived distribution.
- That longer generated responses are necessarily better responses.

Important dataset limitation

CodeAlpaca-20k is a synthetic coding instruction dataset. Its outputs were generated by another language model. This means improvements measured on the same general distribution can reflect adaptation to dataset style as well as improvement in coding behavior.

Important sequence-length limitation

The final training length of 256 tokens was forced by the 6 GB GPU constraint. Some longer examples could not be fully represented in the same way as shorter examples. The generation evaluation also used a 1,024-token ceiling, which the fine-tuned model reached frequently.

Next experiment: semantic correctness

- Extract generated Python safely rather than executing arbitrary raw text.
- Derive or use task-specific test cases where expected behavior is known.
- Compare generated outputs against reference behavior rather than merely checking whether the process exits.
- Categorize failures into algorithmic errors, API misuse, edge cases, missing imports, incomplete solutions, and formatting/extraction failures.
- Where compatible, add HumanEval or MBPP as an external benchmark.

21. Reproducibility Checklist

Core seeds and dataset

SEED = 42 DATA_SEED = 42 Raw records = 20,022 Final examples = 5,338 Train / validation / test = 4,270 / 534 / 534

Core QLoRA configuration

4-bit NF4 double quantization = True compute dtype = float16 LoRA r = 16 LoRA alpha = 32 LoRA dropout = 0.05 target_modules = all-linear max_length = 256 batch_size = 1 gradient_accumulation = 8 epochs = 1 learning_rate = 1e-4 optimizer = paged_adamw_8bit gradient_checkpointing = True assistant_only_loss = True

Expected local artifacts

- data/train/train.jsonl
- data/validation/validation.jsonl
- data/test/test.jsonl
- data/dataset_manifest.json
- model/qwen3.5-4b-python-qlora/
- model/qwen3.5-4b-python-qlora/training_metrics.json
- evaluation/execution_results.json
- evaluation/execution_summary.json
- evaluation/execution_results.csv

Public repository

github.com/raghav7510/Project-02-ML-Coding

Large model files, checkpoints, raw dataset files, and generated artifacts are excluded from the public repository through .gitignore. The repository therefore acts as the auditable source/documentation layer rather than a binary model registry.

22. Final Portfolio Summary

Project-Qwen3.5-4B-QLoRA-Coding demonstrates an end-to-end parameter-efficient fine-tuning workflow under a real consumer-hardware constraint. The project did not treat successful training as the final objective; it documented the data decisions, architecture inspection, memory limits, debugging failures, training metrics, and held-out evaluation.

The most important quantitative result so far is the change in functional execution rate from **22.0%** for the untouched base model to **88.0%** for the QLoRA-adapted model on a 50-example held-out sample. This result supports a strong improvement in executability for the sampled tasks, while the report deliberately avoids calling it semantic code accuracy.

The next research step is therefore not another training run. It is better measurement: task-level correctness, error analysis, and an external coding benchmark where compatible. That distinction is important because a technically credible ML project should separate what the experiment measured from what the model is ultimately claimed to be capable of.

Portfolio-ready technical takeaway

Dimension	Measured evidence
Hardware efficiency	1 epoch completed on 6 GB RTX 4050; peak 5.443 GB allocated
Parameter efficiency	32.46M trainable parameters / 0.766% of total
Data discipline	5,338 examples; zero detected prompt/input overlap across splits
Training	534 steps completed; final train loss 0.08355
Generation	Fine-tuned outputs were substantially longer and hit 1,024-token ceiling more often
Functional behavior	22.0% → 88.0% successful execution on sampled held-out tasks
Scientific limitation	Semantic correctness remains unmeasured

Repository: github.com/raghav7510/Project-02-ML-Coding

Document title: Project-Qwen3.5-4B-QLoRA-Coding — Detailed Technical Report